

Text, image → unstructured

BLOB → Binary Large Object

Weather Predict / Forecast

ML < Supervised learning

ML < Unsupervised

Don't assume anything

- * Supervised Learning means you have a labeled dataset on which a model needs to be built
- * Unsupervised learning means a unlabeled dataset on which the model will be built
- * Semisupervised learning works on a dataset which is a mixture of both labeled & unlabeled

train set	No. of Action Scene	Features/ Input	Class/ Label
1	100	15	Action } Train
2	20	95	Comedy } Train
3	90	5	Action } Val
4	10	85	Comedy } Test
5	50	50	

- Set of input/output → Labeled dataset
- Dataset contains only input → unlabeled dataset
- * Samples will be used to split dataset
- Preparation for exam → Train
- Exam → Test
- Test (weekly) → Validation
- * In test, you have unseen label data
- * Validation → similar to test dataset but it is used during training phase

- For 50-50
- Step 1- Split the dataset into train, test & validation or just train & validation
 - Step 2- From test, compute the distance for each training point

$$\sqrt{6400 + 6400}$$

$$\sqrt{12800}$$

$$20\sqrt{32}$$

$$\frac{1414}{12}$$

Step 3:- Sort the distance in the ascending order

① Compute Euclidean distance

Sl No.	Dist	Team	Test	Class of Team	Dist
	D1	T1	T5	Action	61.03

Step ② Sort the data on basis of Distance

SNo	Dist.	Team	Test	Class of Team	Distance
4	D4	T4	T5	Comedy	53.1507
2	D2	T2	T5	Comedy	54.0833
3	D3	T3	T5	Action	60.208
1	D1	T1	T5	Action	61.0328

$K \rightarrow$ no. of neighbours to be considered while taking decision (K -nearest neighbour)

Determine value of K

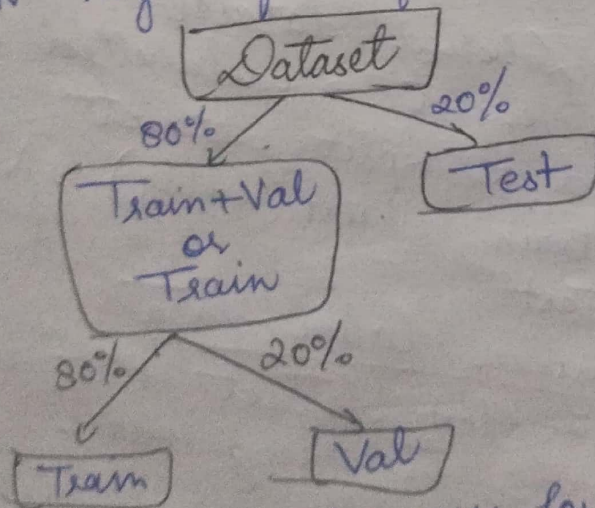
K	Classes
1	{Comedy}
2	{Comedy, Comedy}
3	{Comedy, Comedy, Action}

Majority Vote
Comedy
Comedy
Comedy

Note:- In case of binary classification, K should be odd numbers

* KNN might fail if the dataset is imbalanced.

Dataset $\begin{cases} \text{less (Test \% more)} \\ \text{high (Test \% less)} \end{cases}$



Compute Accuracy for different values of K using

Unsupervised Learning:-

Try to find out pattern within a data or define some rule on the basis of which algorithm

will return some output.
ML uses < distance measure
probability

Clustering

Binary image (0 & 255)
Grey image (0-255)

* Total no. of features to get original image
(assumption - every pixel is different) = No. of pixels in image

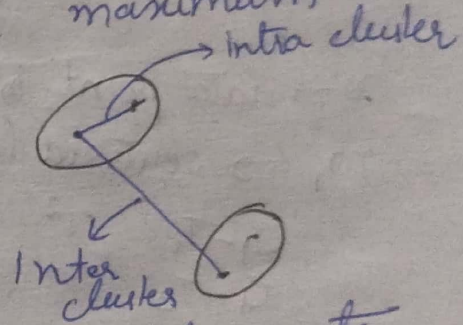
Average → Layman Term

Central Tendency

KMeans → Name of groups → unknown

In clustering, figure out similar properties

* Intra cluster distance should be minimum & inter cluster dist. should be maximum



1) Randomly select seats, the no. of seats should be same as no. of clusters.

Each seat will represent a centroid.
2) Compute the dist. from centroid to all the given points. Assign the points to the cluster which is nearer to them.

3) Recompute the centroid.
4) Repeat step 2 until stopping criteria is met.

$$80\sqrt{2} = 113.13$$

$$10\sqrt{2} = 14.14$$

$$\sqrt{8100 + 4900} = 114.45$$

$$c_1 \rightarrow \begin{matrix} 1 & 100 & 15 \\ 2 & 20 & 95 \end{matrix} \quad \left. \vphantom{\begin{matrix} 1 \\ 2 \end{matrix}} \right\} (60, 55)$$

$$c_2 \rightarrow \begin{matrix} 3 & 90 & 5 \\ 4 & 10 & 85 \end{matrix} \quad \left. \vphantom{\begin{matrix} 3 \\ 4 \end{matrix}} \right\} (50, 45)$$

$$c_1' - 1 \quad 56.5$$

$$c_1' - 2 \quad 56.5$$

$$c_1' - 3 \quad 58.3$$

$$c_1' - 4 \quad 58.3$$

$$c_2' - 1 \quad 59 \quad c_1$$

$$c_2' - 2 \quad 59 \quad c_1$$

$$c_2' - 3 \quad 40 \quad c_2$$

$$c_2' - 4 \quad 40 \quad c_2$$

C_1 100 15
 C_2 20 95

C_1-1 0
 C_1-2 112
 C_1-3 14
 C_1-4 114

C_2-1 113
 C_2-2 0
 C_2-3 114
 C_2-4 14

$C_1' (95, 10)$

$C_2' = (15, 90)$

$C_1'-1$ 7
 $C_1'-2$ 113
 $C_1'-3$ 7
 $C_1'-4$ 113

$C_2'-1$ 113
 $C_2'-2$ 7
 $C_2'-3$ 113
 $C_2'-4$ 7

Drawback of K-Means

* It has problems when clusters are of differing

Sum of Squared Error (SSE)

$$= \sum_{i=1}^K \sum_{x \in C_i} \text{dist}^2(m_i, x)$$

$x \rightarrow$ data point in cluster C_i

$m_i \rightarrow$ centroid (mean) for cluster C_i

C_1-T_1 7²
 C_1-T_3 7²

C_2-T_2 7²
 C_2-T_4 7²

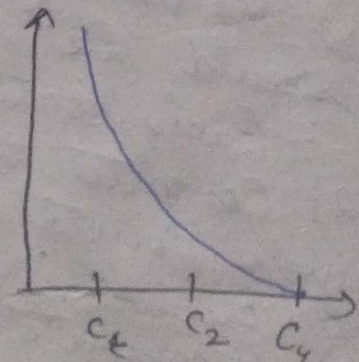
$$\text{SSE} = 4 \times 4 = 16$$

Centroid for only 1 group = $(55, 50)$

CT_1 57.00
 CT_2 57.00
 CT_3 57
 CT_4 57.00

$$(57)^2 \times 4 = 13000$$

* Pick that particular point after which the change is minimum



Stopping Criteria

- * Max. no. of iterations = Stop after 300 iterations
- * Convergence based on centroid movement (Tolerance):
Stop when the centroids move less than 0.0001 b/w iterations.
- * No change in cluster assignments: Stop when no points change their cluster assignment.
- * Convergence in Inertia (Sum of square Distances)

- * K-means has problems when clusters are of differing sizes
- * K-means has problems when the data contains outliers
- * One possible solⁿ is to remove outliers before clustering

- * Forecast is a sub category of prediction
- * Forecast is done for a shorter duration
- * Forecast $\rightarrow$ a predicted value or a prediction with a time stamp

$$f(\text{units}) = -2(\text{units})$$

$$(\hat{y} - y) \rightarrow \text{error}$$

$\bar{x}$ $\rightarrow$ mean

$\hat{y}$ $\rightarrow$ predicted value

$|y - \hat{y}| \rightarrow$ Mean absolute error

$$\text{Error} = \sum (\text{minutes mean} - \text{computer mean})^2$$

Regression Analysis

Some commonly used types of linear forms are:

Simple linear form - Here there is 1 predictor & 1 dependent variable: $f(x) = \beta_0 + \beta_1 x_1 + \epsilon$

Dependent parameters are those for which a prediction will be done on the basis of independent parameters

Multiple linear form - Here there are multiple predictors & 1 dependent variable:

$$f(x) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

Some commonly used types of non-linear forms are:

$$\text{Polynomial form: } f(x) = \beta_0 + \beta_1 x_1^{p_1} + \beta_2 x_2^{p_2} + \dots + \beta_n x_n^{p_n} + \epsilon$$

$$\text{Quadratic form: } f(x) = \beta_0 + \beta_1 x_1^2 + \beta_2 x_1 + \epsilon$$

Logistic form:
$$f(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}} + \epsilon$$

* Logistic regression although is a form of regression analysis but it is used for classification analysis where $\beta_0, \beta_1, \beta_2, \dots, \beta_n$ are said to be regression coefficients

$\epsilon \rightarrow$ Training error
 $\hookrightarrow$ Never used for test data

- * If the dependent parameters are of continuous nature $\rightarrow$ regression analysis
- * If output is having discrete values $\rightarrow$ classification
- * While using this "function" for test data, you should ignore ϵ value.

The best fit model is obtained by solving the linear regression model $\hat{y} = \beta_0 + \beta_1 x + \epsilon$
 To determine the coefficients β_0 & β_1 st error is

minimum:

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$\uparrow$ Cost/Loss Function

Differentiating the eqⁿ $\sum_{i=1}^n (y_i - b_0 - b_1 x_i)^2$ w.r.t b_0 and equating to 0 we get

$$b_1 = \frac{\sum_{i=1}^n (x_i y_i) - n \bar{x} \bar{y}}{\sum_{i=1}^n x_i^2 - n \bar{x}^2}$$

Similarly, differentiating eqⁿ $\sum_{i=1}^n (y_i - b_0 - b_1 x_i)^2$ w.r.t b_0 and equating to 0 we get

$\bar{x}, \bar{y} \rightarrow$ sample mean
 $b_0, b_1 \rightarrow$ regression coefficients

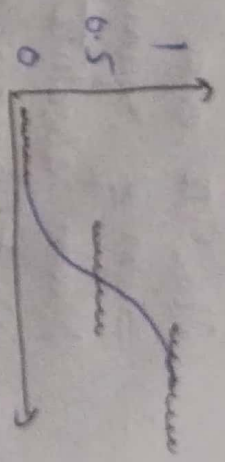
$$\hat{y} = 4.5 + 15x + 348$$

Logistic Regression

Iris dataset (Dependent on sepal & petal size)

Versicolour
Setosa
Virginica

Curse of dimensionality reduces:- which means that in case you are reducing the dimension, some of information will be lost.



$$\text{Odds} \rightarrow \frac{P_{\text{win}}}{P_{\text{lose}}}$$

$$= \frac{P_{\text{lose}}}{P_{\text{win}}}$$

$$P_{\text{win}} = \frac{P_{\text{win}}}{P_{\text{win}} + P_{\text{lose}}}$$

$$P_{\text{lose}} = \frac{P_{\text{lose}}}{P_{\text{win}} + P_{\text{lose}}}$$

Denominator $\uparrow \rightarrow$ Odds $\downarrow$ (0,1)
Numerator $\uparrow \rightarrow$ Odds $\uparrow$ (1, ∞)

How to remove partiality? Infinity

Applying logs

$$\frac{\log \text{ of Odds}}{\text{Odds}} = \frac{P_{\text{win}}}{P_{\text{lose}}} \Rightarrow \log(\text{Odds}) = \log\left(\frac{P_{\text{win}}}{P_{\text{lose}}}\right)$$

$$\Rightarrow \log(\text{Odds}) = \log\left(\frac{P_{\text{win}}}{1 - P_{\text{win}}}\right) = \beta_0 + \beta_1 x \Rightarrow P_{\text{win}} = (1 - P_{\text{win}}) e^{\beta_0 + \beta_1 x}$$

$$\Rightarrow \frac{P_{\text{win}}}{1 - P_{\text{win}}} = e^{\beta_0 + \beta_1 x}$$

$$\Rightarrow P_{\text{win}} (1 + e^{\beta_0 + \beta_1 x}) = e^{\beta_0 + \beta_1 x}$$

$$\Rightarrow P_{\text{win}} = \frac{e^{\beta_0 + \beta_1 x}}{1 + e^{\beta_0 + \beta_1 x}}$$

$$x_1 \xrightarrow{\beta_1} \quad x_2 \xrightarrow{\beta_2} \quad \frac{e^{\beta_0 + \beta_1 x_1 + \beta_2 x_2}}{1 + e^{\beta_0 + \beta_1 x_1 + \beta_2 x_2}} \hat{y}$$

* For regression analysis, you have four ways to
 Absolute error
 Sum of squared errors
 Mean absolute error
 Root mean squared errors
 R^2 , adjusted R^2 , mean squared errors
 Trend

with $x =$

* For classification, you have
 Binary cross entropy
 cross entropy
 Soft max
 KL divergence loss

Functions $\rightarrow$ activation functions sigmoid, tanh, relu, leaky relu, elu, Swish, mish,

(Convention of activation functions, loss functions)

$x_1 \rightarrow \bigcirc$ $\rightarrow$ In ML or DL, the input layer will accept data for processing.

$x_2 \rightarrow \bigcirc$ $\rightarrow$ The no. of neurons in the input layer will be same as no. of features

$\rightarrow$ In the input layer no weights should be assigned

Output
 $\rightarrow$ The no. of neurons in the output layer will depend upon the nature of problem

2) For regression analysis, no. of neurons will be 1

2) For binary classification, no. of neurons may be 1 or 2

For ± 0.2 the no. of neurons is 1, then the loss func. should be binary cross entropy.

3) Multiclass classification will have no. of neurons in the output layer will be same as no. of classes

* For regression analysis, your loss function may be:-

Absolute error

Sum of squared errors

Mean absolute errors

Root mean squared errors

R^2 , adjusted R^2 , mean squared errors

Trend

tanh $x =$

* For classification, you have

Binary cross entropy

cross entropy

Soft max

KL divergence losses

Functions $\rightarrow$ activation functions
sigmoid, tanh, relu, leaky relu
elu, SeLU, mish,

(Derivation of activation functions, loss functions)

x_1 

$\rightarrow$ In ML or DL, the input layer will accept data for processing.

x_2 

Input

$\rightarrow$ The no. of neurons in the input layer will be same as no. of features

$\rightarrow$ In the input layer no weights should be assigned.

Output

$\rightarrow$ The no. of neurons in the output layer will depend upon the nature of problem.

1) For regression analysis, no. of neurons will be 1.

2) For binary classification, no. of neurons may be 1 or 2. If no. of neurons is 1, then the loss function should be binary cross entropy.

3) Multiclass classification will have no. of neurons in the output layer will be same as no. of classes.

$$f(x) = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

$$\sigma(f(x)) = \frac{1}{1 + e^{-f(x)}}$$

$$\sigma \left(\frac{f(x)}{1} \right)$$

* In case of artificial or deep neural network, the activation function $\rightarrow$ non-linear nature

1) Sigmoid:- $\sigma(x) = \frac{1}{1 + e^{-x}}$ Range:- $(0, 1)$

Graph:- S-shaped curve
* Used in binary classification problems

2) Tanh (Hyperbolic Tangent) $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ Range:- $(-1, 1)$

Graph:- S-shaped curve, but symmetric around 0
* Inputs have +ve & negative values

3) ReLU (Rectified Linear Unit)
 $\text{ReLU}(x) = \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases}$ Range:- $[0, \infty)$

Graph:- Linear for $x > 0$, flat for $x \leq 0$
* Used in deep learning models

4) Leaky ReLU
 $\text{Leaky ReLU}(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$ Range:- $(-\infty, \infty)$
 $\alpha \rightarrow$ small constant (eg:- 0.01)

Graph:- Similar to ReLU but with a small slope for $x \leq 0$
* Suitable for models with sparse activations

5) ELU (Exponential Linear Unit)
 $\text{ELU}(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$ Range:- $(-\infty, \infty)$
 $\alpha > 0$ (commonly $\alpha = 1$)

Graph:- Exponential decay for $x \leq 0$ & linear for $x > 0$

6) Gushu:-
 $\text{Gushu}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$ Range:- $(-\infty, \infty)$